Agentic Engineering Fundamentals (45-Min Crash Course)

References & Companions, Chapter 4/4 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/agentic-engineering-crash-course

EN: 8 Concepts, 80% of Real Use. Thesis: software fundamentals matter MORE in the AI era, not less -- an agent's interfaces, names, and boundaries become its inputs.
Roman Urdu: 8 Concepts, 80% of Real Use. Thesis: software fundamentals AI era mein ZYADA matter karte, kam nahi -- agent ke interfaces, names, boundaries uske inputs ban jate hain.

1. Vibe Coding -> Agentic Engineering + Three Constraints

- Software 1.0 (code) -> 2.0 (weights) -> 3.0 (prompting). Vibe coding raises the floor; agentic engineering preserves the CEILING -- for payroll/regulated/multi-user work, vibe coding is malpractice.
- **Smart zone/dumb zone:** attention budget dilutes as the session fills, ~100k-token practical waterline regardless of the advertised window.
- **Memento problem:** models are stateless -- CLEARING beats compaction (compaction carries dumb-zone reasoning forward). **Jagged intelligence:** peaks where RL-verifiable (code/math), valleys elsewhere -- choose on-distribution stacks.

2. Six Failure Modes + Cures

- Misalignment (cure: grill-me), verbose agent/no shared language (cure: CONTEXT.md), weak feedback loops (cure: tdd), ball of mud/shallow modules (cure: improve-codebase-architecture), brain saturation (cure: gray-box), review bottleneck (cure: automated+human review split).

3. End-to-End Workflow + Skills

- Day shift (human-in-loop) / Night shift (AFK, sandboxed), mediated by durable HANDOFF artifacts. Specs are handoff artifacts, not the whole workflow -- they decay.
- Vertical slices (tracer bullets) beat horizontal slicing. Skill = agent reads (on-demand); Tool = agent calls -- portable across Claude Code/OpenCode.

4. Architecture Principles

- Deep modules (small interface, large hidden implementation) beat shallow ones. Test AT THE INTERFACE. Gray-box: design the interface, delegate the implementation.
- 'You can outsource your thinking, but you can't outsource your understanding.' (Karpathy) Agent = tactical (sergeant); human = strategic (which hill, design concept, map).

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Smart zone / dumb zone	Attention budget dilutes as a session fills -- ~100k-token practical waterline.	Session bharne se attention budget dilute hota -- ~100k-token practical waterline.
Memento problem	Models are stateless -- clearing beats compaction (compaction carries bad reasoning forward).	Models stateless hain -- clearing compaction se behtar (compaction bad reasoning aage le jati).
Jagged intelligence	Peaks where RL-verifiable (code/math), valleys elsewhere -- choose on-distribution stacks.	RL-verifiable jagah peaks (code/math), baaki jagah valleys -- on-distribution stacks chuno.
Deep module	Small interface, large hidden implementation -- the architecture goal.	Chhota interface, bara hidden implementation -- architecture ka goal.
Tracer bullet (vertical slice)	A thin end-to-end slice through the system, preferred over horizontal slicing.	System ke through ek thin end-to-end slice, horizontal slicing se behtar.

Quick Self-Check

Question	Answer
Why is vibe coding called 'malpractice' for payroll/regulated/multi-user work?	Vibe coding raises the floor for quick tasks but doesn't preserve the ceiling needed for correctness-critical, high-stakes systems.
Why does clearing context beat compaction in the 'Memento problem'?	Compaction carries forward reasoning formed in the dumb zone, while clearing starts clean instead of compounding degraded reasoning.
What does 'jagged intelligence' imply for choosing a tech stack?	Prefer on-distribution stacks (well-represented in training, like common languages/frameworks) since capability peaks where it's RL-verifiable and has valleys elsewhere.
What's the fix for 'brain saturation' as a failure mode?	Gray-box design -- design the interface yourself and delegate the implementation, rather than holding the whole system in your head.

Where This Leads / Sources

Leads to: Agentic Coding Crash Course (its own prerequisite). Full worked gamification example in Part 6; book's own 18-Q quiz in 11-test-your-understanding; practice drills in projects/.

Source: docs/agentic-engineering-crash-course/ (this repo), content originally via Zia Tutor AI connector.